[MIDWAY REPORT] APPENDIX
IS THE GAP AN INTERFACE GAP? AGENT TOOL INTERFACES FOR DATA CATALOGS

Alex Feng

A ENVIRONMENT, EXAMPLES, EVALUATION AND CONDITIONS

Fixture. Three seed tables (5 customers, 8 orders, 10 order items) feed a staging model and two aggregates (`customer_orders`, `daily_revenue`) through the dbt-duckdb adapter, with schema documentation, dbt tests and a JSON catalog snapshot. A compact `catalog/overrides.json` sidecar injects controlled metadata changes. Each example copies the clean base, applies a debt or change overlay, generates the diff, and builds the project (`dbt seed`, `run`, `parse`), so every condition starts from the same state.

Hygiene contract. The contract (v0.1.0, validated by a JSON Schema) names, for each of the five issue types, its required evidence, allowed and forbidden action classes, and intended verification. Limits: 20 assets, two lineage hops, five findings and 10,000 probed rows. Deleting assets, changing access policies, mutating a live catalog, inventing business semantics and unbounded queries are forbidden.

Examples and ground truth. The four remediation cases are a stale revenue-filter description, wrong customer-order grain, a missing owner, and a legacy copy still marked active and canonical. The four flawed changes are a changed revenue filter, a changed grain without updated grain metadata or tests, a new undocumented model, and a rename without a lifecycle transition. The two clean controls are a documented new model and a documented revenue metric. For each example the manifest records the expected `(issue_type, asset)` findings, required evidence, *sets* of acceptable and unsafe actions, postconditions, and intended checks. Action sets let an escalation count as correct where business meaning is uncertain. In the filter-drift case, for example, proposing a description fix is acceptable but editing the SQL to match stale prose is unsafe. The clean controls are meant to measure spurious findings, although both contain unlabelled gaps.

Evaluation procedure. For each example and condition: (1) copy the base fixture, apply the example's debt or change overlay and build the dbt project; for Codex the build goes to a temporary directory with an opaque name outside the repository, and the example file, the marker file that names the example, the pre-change copy and the build log are removed, leaving `project/` (with dbt's `target/`), `warehouse.duckdb` and, for change review, `input.diff`; (2) run the condition once on that state with its recorded prompt and settings; (3) save its findings with their evidence and actions; Codex's answer, which cites file paths or commands rather than evidence IDs, is validated against its output schema and converted to the steward's plan form, each citation becoming an evidence entry; (4) for Codex, run the post-run checks below; (5) reduce findings to `(issue_type, asset)` pairs, match them exactly to the manifest and count TP, FP and FN; (6) count the example as successful only if the run completes without missing or extra findings, forbidden action classes, or a failed verification; and (7) inspect the saved trace. Precision is TP/(TP+FP) and recall is TP/(TP+FN); micro scores sum the counts over examples before dividing. The scorer (`midpoint-score-v1`, unchanged for Codex) checks that evidence resolves, not that it supports the claim. It checks action classes, not example-specific acceptable actions or postconditions. The AI-assisted trace audit examines those distinctions separately.

Post-run checks for Codex. After each run the harness (a) diffs the project before and after and records every changed file; (b) applies the steward's patch verification to every patch finding: the override file changed that field, the merged catalog field equals the value Codex declared, and `dbt parse` succeeds on Codex's final workspace, so a claimed patch with no file change fails; (c) checks that only `catalog/overrides.json` changed and that every changed field is declared by a patch finding; and (d) scans the event stream for the repository path, example IDs, label file names, paths outside the workspace, parent-directory reads, and web or connector use. A failure in (b) or (c) is a verification failure; (d) flags a run for the audit without changing its score. For Codex,

evidence validity is the share of citations whose file exists in the workspace or whose command ran successfully; it is not the steward's check that evidence IDs resolve, so it is reported separately.

Shared by all three conditions. The example fixtures and their build; the trigger and asset scope; the contract's five issue types with their required evidence and allowed and forbidden actions; the five-finding cap; the patch surface (five fields of an asset in scope in `catalog/overrides.json`); the instruction not to use ground truth or invent business meaning; and the scorer.

Different between the steward and Codex. *Inputs:* the steward receives a precomputed evidence bundle and may call nine read-only tools; Codex reads the same project, warehouse and diff itself with a shell, `dbt`, and Python with DuckDB from a tool environment outside the repository. *Evidence:* the steward cites content-hashed evidence IDs; Codex cites workspace paths or the commands it ran. *Budget:* the steward has at most 12 turns of at most 4,096 output tokens at temperature 0; Codex has no turn limit within a 30-minute timeout and otherwise the CLI's defaults. *Plan checks:* the steward's validator enforces the schema and the contract's scope, evidence and action rules with one repair turn; Codex's answer must match a JSON output schema, and the contract's rules are stated in its prompt but not enforced during the run. *Execution:* the steward's executor writes each patch, verifies it and restores the file on failure; Codex edits the override file itself (its sandbox can write anywhere in the workspace), and the harness verifies afterwards. *Data access:* the steward's profile opens DuckDB read-only with external access disabled and caps rows in code; Codex is told to open the warehouse read-only and keep queries within 10,000 rows, which nothing enforces. *Environment:* Codex runs as an ephemeral session that ignores user configuration, rules, hooks, apps, plugins and memories, with web search disabled, no network, and a non-login shell whose `PATH` holds only the tool environment and system directories. *Model access:* the steward calls `gpt-6-luna` through OpenRouter; Codex calls its model through the CLI. *Records:* two steward records (both revenue-filter examples) replay an earlier identical run; every Codex record is a fresh run.

Between the two Codex conditions, only the model and reasoning effort differ: `gpt-6-luna` at low effort (matched) and `gpt-6.1-sol` at medium effort (frontier). Frozen configurations: steward, `experiments/configs/midpoint_v1.yaml`; Codex, `experiments/configs/codex_baseline_v1.yaml`.

B System Details

Evidence bundle. For each scoped asset (at most 20) the steward collects catalog metadata (the snapshot merged with the override file), dbt schema documentation and tests, model SQL, parsed SQL facts (sources, filters, joins, aggregates, grouping, grain candidates, select expressions), the dbt manifest node, one-hop lineage, and a bounded profile. Change review adds the diff. The profile reads at most 10,000 rows from a read-only DuckDB connection with external access disabled and reports per-column null, distinct and duplicate counts. Every item has an ID derived from a hash of its kind, asset, source and content, plus a provenance pointer. Absent metadata is recorded as explicit `present: false` evidence. The prompt contains only the trigger, asset scope, evidence and collection errors, never the example ID or labels.

Agent loop and tools. The model works through a JSON reply protocol for at most 12 turns. Three consecutive malformed replies end the run. During planning it may call nine read-only tools, whose arguments are checked against JSON Schemas: catalog metadata, dbt schema, model SQL, SQL facts, manifest node, lineage (up to two hops), a bounded profile of chosen columns, the change diff, and a file read confined to the example project (at most 50 kB). No write tools are registered.

Validator and action policy. The final plan must be JSON that matches the steward plan schema, with at most five findings. Each finding must name an in-scope asset and an issue type in the contract, choose an action class that the contract allows for that issue type, cite only evidence IDs from this run, and cite every evidence kind the issue type requires. A failing plan is returned once with the error; a second failure ends the run with no action. Only missing required metadata and SQL/metadata drift allow a patch; grain, reliability and lifecycle issues allow only a proposal or a report. Because these rules key on the model's own issue label, a mislabelled finding can bypass them.

Execution, verification and ledger. A patch may set one of five fields (description, owner, lifecycle, grain, canonical) in the example's `catalog/overrides.json`. The steward saves a unified diff and writes the file. It then re-reads the merged catalog metadata to check that the field equals the value the plan declared, and runs `dbt parse`. If either check fails, it restores the file and records a verification failure. A proposal or report is checked only for an allowed action class and resolved evidence IDs. Each outcome, with its finding, checks and patch path, is appended to `steward_ledger.jsonl` alongside a run summary. A finding already recorded as verified is not executed again. The contract's per-issue verification checks (for example, comparing metadata with parsed SQL, or rerunning a uniqueness probe) are not implemented.

C PIVOT EVIDENCE

Example paths below are relative to the pilot folder `experiments/results/midpoint_v1_2_pilot/`.

Reply format. In the preliminary run (2 examples, config 1.1.0), `render_tools()` showed both reply shapes as one object keyed by `tool_call` and `final_answer`, and the model copied that wrapper; 7 of 8 rejected replies had this cause, and one run repeated it three times and stopped before reading the diff. The raw records of that run were not kept; the diagnosis is recorded in commit `d2a547b` and `writeups/midway_report/.agents/agent_notes.md` (2026-11-08). The fix, in config 1.2.0, shows two plain example objects, makes protocol errors say what to send instead, and gives both model conditions one identical repair turn for an invalid final plan. Prompts were frozen after a rerun under 1.2.0 and before the pilot.

Raw-tool exploration. The first reply in `change_daily_revenue_filter/generic.json` holds several `read_diff` and `read_file` calls in one message and is rejected as more than one JSON object; the second reply is a final plan with no findings whose summary says the evidence "only lists project files". The only evidence in the record is the project listing. In `rem_daily_revenue_missing_owner/generic.json`, the first reply held at least 13 tool calls with invented results and a draft answer; no tool ran, and the final plan reported three issues about a change that does not exist, citing the project-listing ID after the repair turn. The scorer gave it evidence validity 1.0 because it checks only that IDs resolve. Seven of its ten pilot runs hit a rejection, against two for the steward (`analysis.md`).

Label choice. In `rem_customer_orders_wrong_grain/steward.json`, the steward's single finding is `sql_metadata_drift` on `customer_orders`, with a patch setting the catalog grain to `customer_id` and evidence from the catalog, dbt schema, SQL, parsed SQL facts and manifest but no uniqueness probe. The value is correct and was verified as written, but the manifest expects `grain_or_join_drift` with a proposal and uniqueness evidence, so the example scores 0/1/1 and fails. The validator allowed the patch because its rules key on the model's own label.

Unmatched baseline. The first Codex runs used Codex CLI 0.167.0 with `gpt-6.1-terra` at medium effort, while the steward pilot used `gpt-6-luna` at low effort; they are recorded in `experiments/results/codex_midway_two_case/README.md`. The prompt listed the five issue types but not their required evidence or allowed actions, and Codex's path-based findings were mapped to manifest labels by hand. Both agents scored 1/0/0 on the revenue-filter case and 2/2/0 on the changed-grain case. The matched and frontier baseline that replaced these runs is reported in the Results section of the report, with its records and trace audit in `experiments/results/codex_baseline_v1/`.